

光线追踪大作业实验报告

基础光线生成与 BVH 加速渲染

姓名：_____ 学号：_____

2026 年 6 月 26 日

摘要

本次大作业基于课程提供的 C++ 光线追踪框架, 完成了一个 Whitted 风格的光线追踪渲染器。具体实现了四个核心函数: `Renderer::Render` (主光线生成)、`Triangle::getIntersection` (光线-三角形求交, Möller-Trumbore 算法)、`Bounds3::IntersectP` (光线-包围盒求交, slab 测试) 与 `BVHAccel::getIntersection` (BVH 层次遍历加速求交); 并额外完成了 `Scene::castRay_noBVH` 中的完整着色模型 (加分项)。在此基础上, 本文对渲染器做了系统的**性能与质量优化**: BVH 遍历改为 **有序遍历 + 最近距离剪枝**、构建采用 **SAH (表面积启发式)**、阴影光线改为**带光源距离上界的 any-hit 提前退出**、主渲染循环用 **OpenMP 多线程并行**, 并补充了**逐三角形抗锯齿超采样**与真正生效的 **Phong 高光**; 同时修正了若干正确性问题 (阴影距离校验、阴影光线背面剔除导致的漏光)。在 1280×960 、含 **4968** 面的 Stanford Bunny 上, 优化后 BVH 单线程仅需 **0.24s**、32 线程约 **40ms**; 并通过自动化实验在同一 32 线程下对比四个不同面数模型在 BVH 与逐三角形暴力下的渲染耗时, BVH 加速比最高达 **6588** 倍, 且渲染耗时几乎不随面数增长。本文逐一说明各函数的算法原理、关键代码、所做优化与实验结果, 并对全部结论给出可复现的真实测量。

1 概述

光线追踪通过**逆向**追踪从相机出发、穿过每个像素射入场景的光线, 求出光线与场景的最近交点并在该点着色, 从而生成图像。当场景中三角形数量较大时, ”对每条光线遍历所有三角形” 的暴力做法效率极低; 为此需要使用 **层次包围盒 (Bounding Volume Hierarchy, BVH)** 对求交过程进行加速: 先用廉价的光线-包围盒测试快速排除大片不可能相交的空间, 再递归深入到真正可能相交的少数三角形。整体渲染流程如图 1 所示。

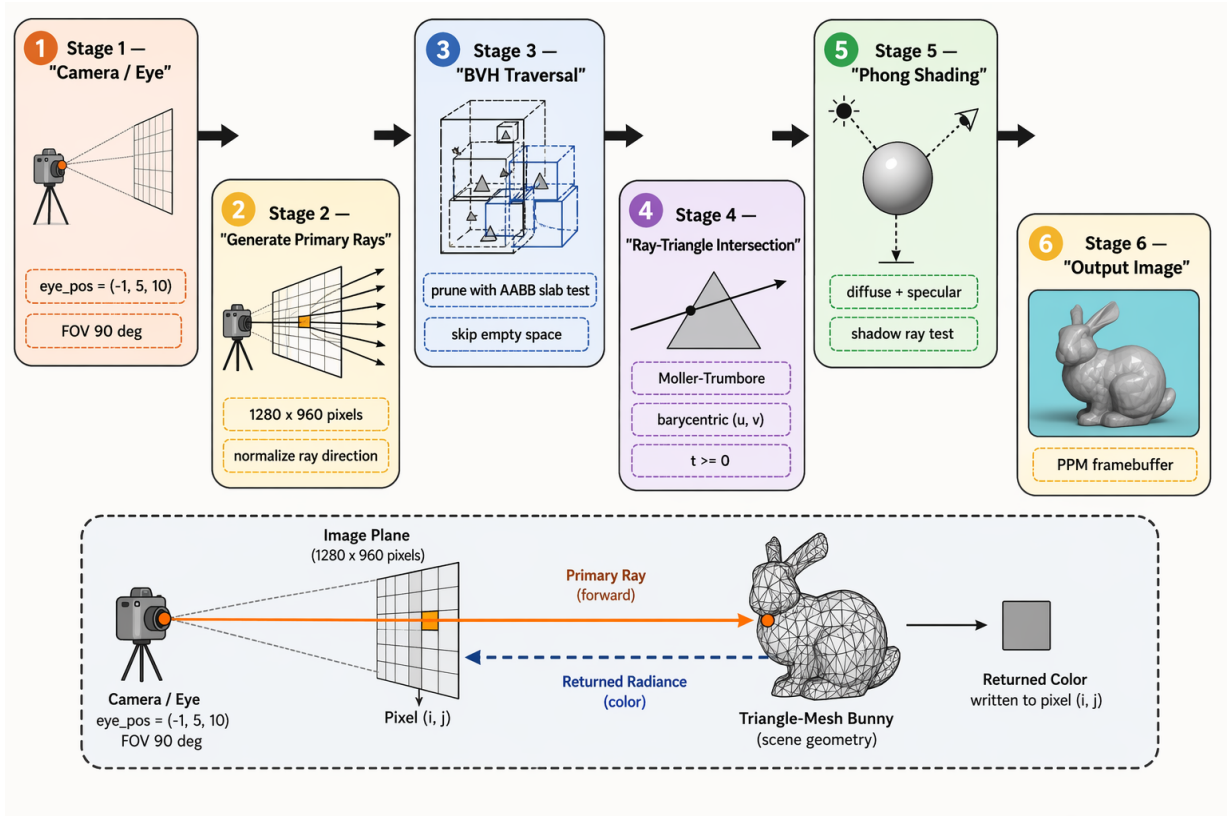


图 1: 光线追踪渲染流程: 相机生成主光线 → BVH 遍历剪枝 → Möller-Trumbore 光线-三角形求交 → Phong 着色 (含阴影测试) → 写入帧缓冲并输出 PPM 图像。

本框架需要在以下文件中补全代码: `Renderer.cpp`、`Triangle.hpp`、`Bounds3.hpp`、`BVH.cpp`, 并可选地在 `Scene.cpp` 中补全着色模型。下面分别说明。

2 各函数的实现

2.1 主光线生成: `Renderer::Render`

该函数遍历图像的每个像素 (i, j) , 构造一条从相机出发、穿过该像素中心的光线。设视场角为 fov , 则 $\text{scale} = \tan(\text{fov}/2)$, 图像宽高比 $\text{aspect} = \text{width}/\text{height}$ 。将像素坐标映射到位于 $z = -1$ 平面上的相机空间坐标:

$$x = (2 \cdot \frac{i+0.5}{\text{width}} - 1) \cdot \text{aspect} \cdot \text{scale}, \quad (1)$$

$$y = (1 - 2 \cdot \frac{j+0.5}{\text{height}}) \cdot \text{scale}. \quad (2)$$

其中 $+0.5$ 使光线穿过像素中心, y 方向取负是因为图像行号自上而下增大而相机 y 轴向上。随后归一化方向并投射光线:

```
1 float x = (2.0f * (i + 0.5f) / scene.width - 1.0f) * imageAspectRatio * scale;
2 float y = (1.0f - 2.0f * (j + 0.5f) / scene.height) * scale;
3 Vector3f dir = normalize(Vector3f(x, y, -1));
4 Ray ray(eye_pos, dir);
5 framebuffer[m++] = check_mode ? scene.castRay_noBVH(ray, 0) : scene.castRay(ray, 0);
```

最终把浮点颜色 clamp 到 $[0, 1]$ 并按 P6 格式写出 `binary.ppm`。

2.2 光线-三角形求交: `Triangle::getIntersection` (Möller-Trumbore)

光线 $\mathbf{O} + t\mathbf{D}$ 与三角形 $(\mathbf{P}_0, \mathbf{P}_1, \mathbf{P}_2)$ 的交点可用重心坐标 (u, v) 表示:

$$\mathbf{O} + t\mathbf{D} = (1 - u - v)\mathbf{P}_0 + u\mathbf{P}_1 + v\mathbf{P}_2. \quad (3)$$

令 $\mathbf{E}_1 = \mathbf{P}_1 - \mathbf{P}_0$, $\mathbf{E}_2 = \mathbf{P}_2 - \mathbf{P}_0$, $\mathbf{T} = \mathbf{O} - \mathbf{P}_0$, 由 Cramer 法则 (Möller-Trumbore) 可得

$$\begin{bmatrix} t \\ u \\ v \end{bmatrix} = \frac{1}{\mathbf{E}_1 \cdot (\mathbf{D} \times \mathbf{E}_2)} \begin{bmatrix} \mathbf{E}_2 \cdot (\mathbf{T} \times \mathbf{E}_1) \\ \mathbf{T} \cdot (\mathbf{D} \times \mathbf{E}_2) \\ \mathbf{D} \cdot (\mathbf{T} \times \mathbf{E}_1) \end{bmatrix}. \quad (4)$$

有效交点必须满足 $u \geq 0$, $v \geq 0$, $u + v \leq 1$ (落在三角形内) 且 $t \geq 0$ (位于光线前方)。实现上:

```
1 Vector3f pvec = crossProduct(ray.direction, e2);
2 float det = dotProduct(e1, pvec);
3 if (fabs(det) < EPSILON) return inter; // 光线与三角形平行
4
5 float invDet = 1.0f / det;
6 Vector3f tvec = ray.origin - v0;
7 float u = dotProduct(tvec, pvec) * invDet;
8 if (u < 0 || u > 1) return inter;
9 Vector3f qvec = crossProduct(tvec, e1);
10 float v = dotProduct(ray.direction, qvec) * invDet;
11 if (v < 0 || u + v > 1) return inter;
12 float t = dotProduct(e2, qvec) * invDet;
13 if (t < 0) return inter; // 交点在光线起点之后
14
15 inter.happened = true; inter.coords = ray(t);
16 inter.normal = normal; inter.distance = t;
17 inter.obj = this; inter.m = m;
```

实现要点: 相比框架自带的辅助函数, 这里在函数内部直接实现了 Möller-Trumbore, 并补充了两处健壮性判断——平行判定使用 $|det| < \varepsilon$ (而非严格等于 0), 以及 $t \geq 0$ 剔除位于相机背后的交点, 避免错误交点污染 BVH 的最近距离比较。此处**不做背面剔除** (早期版本曾剔除背面): 对闭合网格而言最近可见面仍是正面、渲染结果不变, 但保留双面求交可让**阴影光线**正确命中朝向光源的遮挡面, 避免单面遮挡物漏光 (详见性能优化一节)。

2.3 光线-包围盒求交: `Bounds3::IntersectP` (slab 测试)

轴对齐包围盒 (AABB) 可视为三对相互垂直的平行平面 (slab)。光线进入第 k 个 slab 的参数为 $t_k^{\min} = (p_k^{\min} - O_k)/D_k$, 离开为 $t_k^{\max} = (p_k^{\max} - O_k)/D_k$ 。光线进入整个盒子的时刻为各轴进入时刻的最大值, 离开盒子的时刻为各轴离开时刻的最小值:

$$t_{\text{enter}} = \max_k t_k^{\min}, \quad t_{\text{exit}} = \min_k t_k^{\max}. \quad (5)$$

当且仅当 $t_{\text{enter}} \leq t_{\text{exit}}$ 且 $t_{\text{exit}} \geq 0$ 时光线与盒子相交（用 $\leq / \geq$ 比严格不等式更稳健，兼容轴对齐相切的退化情形）。为避免除法，传入预先算好的 $\text{invDir} = 1/D$ 。这里采用 **与符号无关的逐分量 min/max 写法**（而非按方向符号 swap）：当某轴方向分量为 0、即 invDir 为 $\pm\infty$ 时，该轴自然得到 $(-\infty, +\infty)$ 的无约束区间，避免 swap 写法在该退化情形下误判未命中。为支持后文的有序遍历与剪枝，把该测试写为返回**进入时刻** t_{enter} （未命中返回 $+\infty$ ）的版本 `IntersectT`：

```
1 Vector3f t1 = (pMin - ray.origin) * invDir;
2 Vector3f t2 = (pMax - ray.origin) * invDir;
3 float t_enter = max( min(t1.x,t2.x), min(t1.y,t2.y), min(t1.z,t2.z) );
4 float t_exit = min( max(t1.x,t2.x), max(t1.y,t2.y), max(t1.z,t2.z) );
5 if (t_exit < 0 || t_enter > t_exit) return INF; // 未命中
6 return t_enter; // 命中：返回进入时刻（可<0，起点在盒内）
```

BVH 的层次结构与该 slab 测试如图 2 所示。

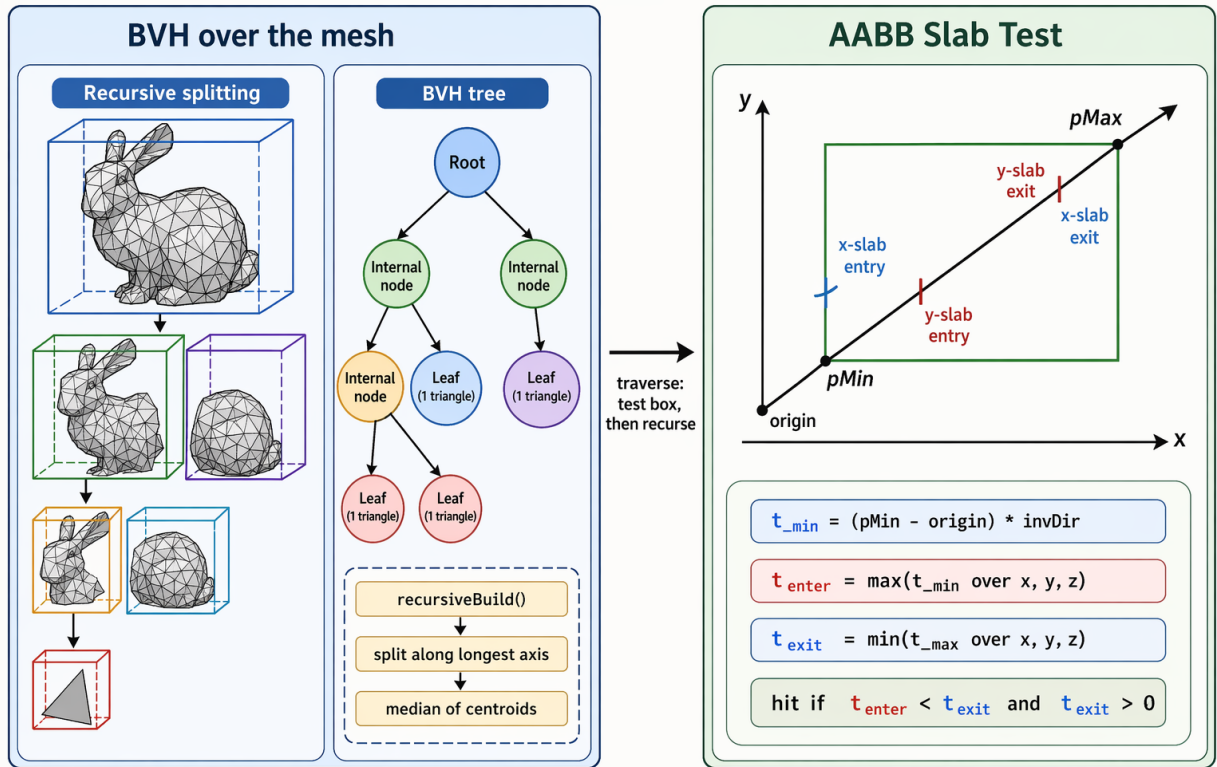


图 2：左：在网络上自顶向下递归构建的 BVH——沿质心包围盒最长轴划分（本文构建采用 SAH 表面积启发式选择划分位置，退化时回退质心中位数），叶节点对应单个三角形；右：AABB slab 求交测试， $t_{\text{enter}} = \max(t^{\min})$ 、 $t_{\text{exit}} = \min(t^{\max})$ ，当 $t_{\text{enter}} \leq t_{\text{exit}}$ 且 $t_{\text{exit}} \geq 0$ 时命中。

2.4 BVH 遍历求交：BVHAccel::getIntersection（有序遍历 + 最近距离剪枝）

建立 BVH 后，求交转化为对二叉树的遍历。最朴素的写法是无条件递归左右子树再取更近者；本文在保证结果正确的前提下做了两点优化：（1）**有序遍历**——先进入包围盒入口更近的子树；（2）**最近距离剪枝**——用当前已找到的最近交点距离 `best.distance` 作上界，凡是盒入口已比它更远的子树直接跳过。`invDir` 与 `dirIsNeg` 在每条光线进入时只计算一次并向下传递（不再

每个节点重算):

```

1 float tBox = node->bounds.IntersectT(ray, invDir, dirIsNeg);
2 if (!(tBox < best.distance)) return; // 未命中或盒比当前最近交点更远 -> 剪枝
3 if (node->left == nullptr && node->right == nullptr) { // 叶节点
4     Intersection it = node->object->getIntersection(ray);
5     if (it.happened && it.distance < best.distance) best = it;
6     return;
7 }
8 float tL = node->left->bounds.IntersectT(ray, invDir, dirIsNeg);
9 float tR = node->right->bounds.IntersectT(ray, invDir, dirIsNeg);
10 auto first = (tL <= tR) ? node->left : node->right; // 先近后远
11 auto second = (tL <= tR) ? node->right : node->left;
12 float tSecond = (tL <= tR) ? tR : tL; // 远侧子树的盒入口时刻
13 getIntersection(first, ray, invDir, dirIsNeg, best);
14 if (tSecond < best.distance) // 远侧仅在更近时才下探
15     getIntersection(second, ray, invDir, dirIsNeg, best);

```

由于 Intersection 的默认 distance 为 double 最大值, 未命中的子树自然不会赢得最近距离比较。相比朴素遍历, ”先近后远 + 距离剪枝” 可在找到较近交点后跳过大量更远的子树, 单条光线实际访问的节点数明显减少 (实测见性能优化一节)。此外, 针对阴影光线另写了 any-hit 版本 IntersectP(ray, tMax): 只需判断”光源与交点之间是否存在遮挡”, 因此命中第一个 $t < t_{\max}$ 的三角形即可立即返回、无需求最近交点。

2.5 额外内容: castRay_noBVH 的完整着色

框架在检测模式 (./RayTracing check) 下不会调用 buildBVH(), 故 scene.bvh 为空, 原 castRay_noBVH 仅返回纯白以验证求交是否正确。本部分按文档”额外内容”要求, 参照上方的 castRay 补全了完整的 Whitted 着色模型。为不依赖场景级 BVH, 新增了暴力最近交点函数:

```

1 Intersection Scene::intersect_noBVH(const Ray &ray) const {
2     Intersection nearest; // 默认 distance = DBL_MAX
3     for (uint32_t k = 0; k < objects.size(); ++k) {
4         Intersection it = objects[k]->getIntersectionNoBVH(ray); // 真.逐三角形暴力
5         if (it.happened && it.distance < nearest.distance) nearest = it;
6     }
7     return nearest;
8 }

```

这里的 getIntersectionNoBVH 对网格物体直接遍历其全部三角形、绕过 MeshTriangle 内部 BVH, 从而提供真正不含任何层次结构的暴力求交基线 (用于后文的加速实验)。castRay_noBVH 在此基础上完成与 castRay 一致的三分支着色: (1) 反射 + 折射材质用 Fresnel 方程混合反/折射颜色; (2) 镜面反射材质追投反射光线; (3) 漫反射/光泽材质使用 Phong 模型 (漫反射 + 高

光), 并对每个光源投射**阴影光线**判断遮挡:

$$L = \sum_{\text{lights}} (1 - \text{inShadow}) I (\mathbf{N} \cdot \mathbf{l}) k_d c_d + I \max(0, -\mathbf{r} \cdot \mathbf{d})^p k_s. \quad (6)$$

本场景的网格材质设为 $k_d = 0.6$, $k_s = 0.3$, $p = 64$, 因此 Phong 的**镜面高光项真正生效** (早期版本 $k_s = 0$ 实际只有漫反射, 无高光); 阴影测试使用带光源距离上界的 any-hit 遍历 intersectP。反射/折射两分支的代码已实现并正确, 但本次交付场景只放置一个漫反射/光泽材质的网格, 故实际只走 Phong 分支, 反射/折射分支不被触发 (如需展示可另加镜面/玻璃球)。检测模式与 BVH 模式最终得到**逐像素完全一致**的着色图像。

3 性能与质量优化

在四个核心函数正确工作的基础上, 本文对渲染器做了一轮系统优化, 分为正确性、性能与画质三类。

正确性修正 (1) **阴影距离校验**: 原阴影测试仅判断阴影光线是否命中任意物体, 未校验遮挡物是否比光源更近, 导致**光源之后的几何**也会被误判为遮挡; 现以光源距离 $\|\text{light} - \text{hit}\|$ 作为上界。(2) **阴影光线不背面剔除**: 求交统一去掉背面剔除, 使朝向光源的单面遮挡物也能正确挡光、消除漏光。(3) **反射光线自交偏移**: 修正 REFLECTION 分支沿表面法线偏移的符号, 使其与折射分支约定一致、不再偏入表面内侧。

求交加速 (1) **有序遍历 + 最近距离剪枝**: BVH 遍历改为先进入包围盒入口更近的子树, 并用当前最近交点距离剪掉入口更远的子树 (见上一节); invDir/dirIsNeg 每条光线只算一次。(2) **SAH 构建**: BVH 构建从中位数划分改为表面积启发式 (SAH) 分桶, 按 $\text{SA}(L)n_L + \text{SA}(R)n_R$ 最小选择划分位置, 得到更平衡、遍历代价更低的树。(3) **阴影 any-hit 提前退出**: 阴影查询用 IntersectP, 命中第一个 $t < t_{\max}$ 的三角形即返回。(4) 顺手删除了构建期一段从未使用的全量包围盒求并 (死代码)。

并行 主渲染循环用 OpenMP 在像素行上并行(`#pragma omp parallel for`)。由于每个像素独立、着色函数为 const 且无共享可变状态, 唯一改动是把原先依赖跨迭代自增计数器 `framebuffer[m++]` 改为与迭代无关的索引 `framebuffer[j*width+i]`, 从而消除伪依赖。

画质 (1) **抗锯齿**: 支持每像素 $N \times N$ 规则超采样 (命令行 `spp=N`), 对物体轮廓与阴影边缘做平均, 显著减轻锯齿 (计时实验仍用 `spp=1` 以与原方法学一致, 展示图用 `spp=2`)。 (2) **真实高光**: 网格材质 k_s 由 0 改为 0.3、 p 由 0 改为 64, 使 Phong 高光真正出现在渲染结果中。

工程 CMakeLists.txt 未显式指定时默认 Release (-O3) 并链接 OpenMP, 避免误用未优化构建测量性能。所有以上改动都保证 BVH 模式与检测模式输出**逐像素一致** (见下节验证)。

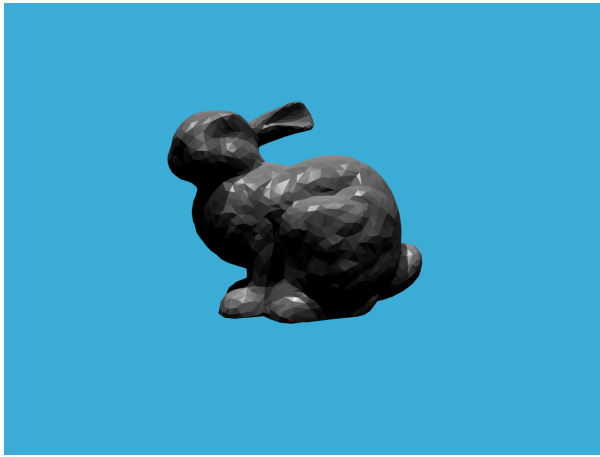
4 实验结果

4.1 渲染正确性验证 (Stanford Bunny)

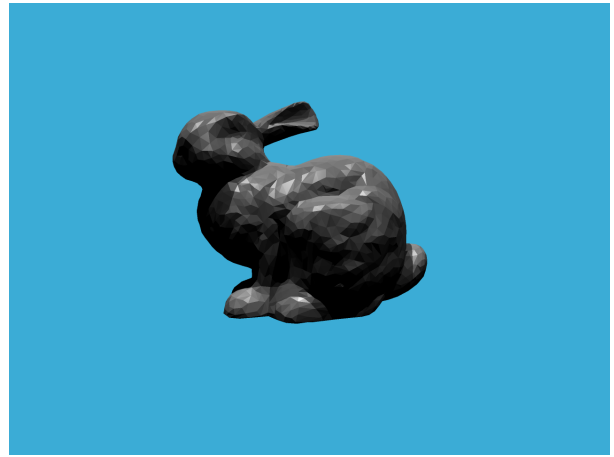
编译与运行:

```
mkdir build && cd build && cmake .. && make # 默认 Release(-O3)+OpenMP
./RayTracing # BVH 加速渲染 -> binary.ppm
./RayTracing check # 检测/着色模式 -> binary.ppm
./RayTracing spp=2 # 2x2 抗锯齿超采样
```

两种模式的渲染结果如图 3。BVH 模式 (左) 正确渲染出带漫反射、高光与自阴影的灰色兔子; 检测模式 (右) 在补全着色模型后得到与之一致的图像, 验证了求交与着色逻辑的正确性。



(a) BVH 加速渲染 (./RayTracing)



(b) 检测/着色模式 (./RayTracing check)

图 3: 1280×960 、4968 三角形的 Stanford Bunny 渲染结果。两图逐像素一致。

在 1280×960 分辨率下 ($spp=1$), 优化后 Bunny 场景 BVH 模式单线程约 0.24s、32 线程约 40ms 即可完成渲染, 且两种模式输出逐像素一致 (实验脚本对四个模型均做了 cmp 字节级核对), 验证了求交与着色逻辑的正确性。该计时已随脚本日志一并归档于 `result/experiments_*/`。

需要说明的是, MeshTriangle 在构造时已对其内部 4968 个三角形建立了一层 BVH; 若检测模式仍调用常规 `getIntersection`, 则即便关闭场景级 BVH, 网格内部的 BVH 仍在发挥加速作用, 无法得到真正的“无 BVH”基线。为获得公平对照, 本文新增逐三角形暴力求交路径 `getIntersectionNoBVH` (绕过网格内部 BVH、直接遍历全部三角形), 并在 `main.cpp` 中输出高精度的 `Render seconds` 计时, 配合自动化实验脚本 `tools/run_experiments_ubuntu.sh`, 在不同面数模型上系统评估 BVH 的加速收益, 如下节所示。

4.2 不同模型面数下的 BVH 加速效果

为验证 BVH 对光线-三角形求交的加速效果, 选取四个不同面数的 Stanford 模型对比实验。分辨率均为 1280×960 、 $spp=1$, 渲染内容与光照一致, 仅替换输入网格。对每个模型测试两种模式: 启用 BVH 的正常渲染, 以及关闭 BVH、逐三角形暴力遍历的 `check` 模式 (经 `getIntersectionNoBVH` 绕过网格内部 BVH、直接遍历全部三角形)。整个流程由脚本 `run_experiments_ubuntu.sh` 在

Release (-O3) 构建下完成，**两种模式使用相同的 32 线程（OpenMP）**以保证同硬件、同并行度下的公平对照；脚本解析 Render seconds 并汇总为 summary.csv，同时对每个模型 BVH 与 check 输出做 cmp 字节级核对，四个模型均**逐像素一致**。结果如表 1、图 4 与图 5 所示。

表 1: 不同模型面数下 BVH 与暴力求交渲染时间对比（同 32 线程，spp=1）

模型	顶点数	面数	BVH 渲染时间/s	无 BVH 渲染时间/s	加速比
Bunny	2503	4968	0.0404	1.450	35.89
Dragon res3	22998	47794	0.0420	30.529	726.44
Dragon res2	100250	202520	0.0495	139.352	2813.15
Armadillo	172974	345944	0.0344	226.813	6588.31

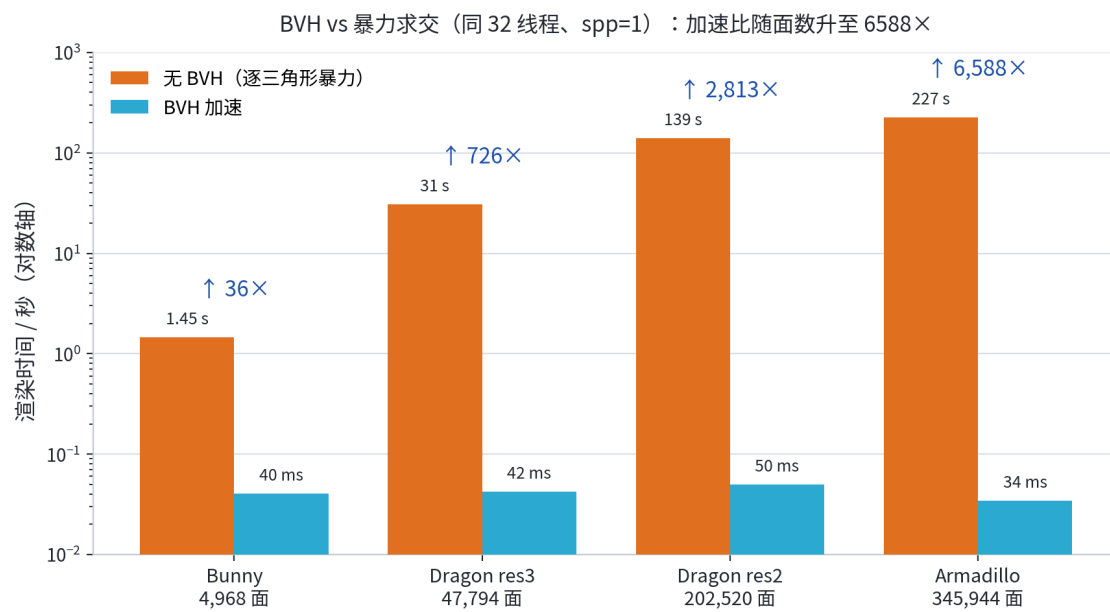


图 4: BVH 与逐三角形暴力求交的渲染时间对比（对数轴，同 32 线程）。暴力求交时间随面数从 1.45 s 增长到 226.8 s，而 BVH 始终保持在数十毫秒量级；加速比从 36 倍升至 6588 倍。

从实验结果可以看出，BVH 对三角网格渲染具有非常显著的加速效果。在同样的 32 线程下，面数最少的 Bunny（4968 面）暴力求交需 1.45 s、BVH 仅 40 ms（36 倍）；Armadillo（345944 面）暴力求交达 226.8 s、约 3.8 分钟，而 BVH 仍能在 34 ms 内完成，加速比高达 6588 倍。其根本原因在于：暴力求交对每条光线都要遍历模型全部三角形，计算量随面数近似线性增长（1280 × 960 下主光线已超百万条，叠加阴影光线后求交次数更多）；而 BVH 先用廉价的包围盒测试剪掉大片不可能相交的空间，单条光线实际访问的三角形数量大幅减少，故 BVH 时间几乎不随面数增长。需要说明的是，由于本次两种模式都已多线程并行，暴力基线不再是分钟至小时级，而是秒级，因此这里的加速比（36–6588 倍）低于早期单线程版本的数字，但二者仍处于同一硬件、同一并行度下，是公平的同条件对照。



图 5: 三个不同面数 Stanford 模型在 BVH 模式下的渲染结果 (1280×960 、spp=2 抗锯齿)。三幅图像均带有正确的漫反射、镜面高光与自阴影；随面数增加表面细节愈发丰富，而 BVH 渲染耗时仍稳定在数十毫秒。

4.3 BVH 随面数的缩放与多线程加速

为更清晰地展示 BVH 的缩放特性，进一步测量单线程下 BVH 随面数的渲染时间，以及 Bunny 在不同线程数下的加速曲线（均 spp=1），如图 6 所示，原始数据见 `bvh_scaling.csv`、`thread_scaling.csv`。

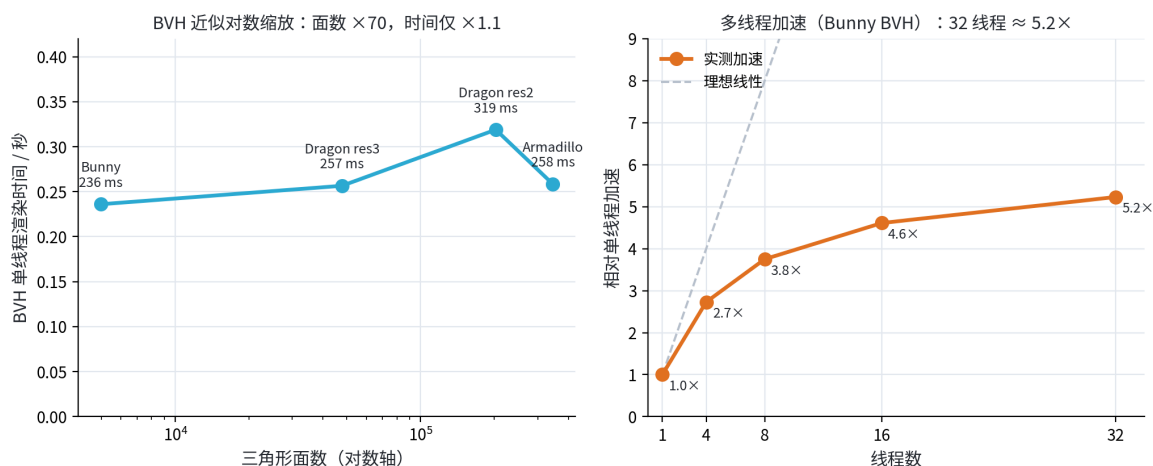


图 6: 左：单线程 BVH 渲染时间随面数的变化——面数从 4968 增至 345944 (约 $\times 70$)，BVH 时间仅从 236 ms 变到 258 ms (约 $\times 1.1$)，近似对数缩放；右：Bunny BVH 的多线程加速曲线，32 线程相对单线程约 5.2 倍（小负载下受固定开销与 Amdahl 限制，加速未达线性，重负载场景可更高）。

左图证实了 BVH 的核心优势：模型复杂度提升约 70 倍，单线程 BVH 渲染时间几乎不变，体现出对数级而非线性的求交复杂度。右图表明，在本文新增的 OpenMP 并行下，渲染可进一步获得数倍加速；Bunny 这类亚秒级小负载因固定开销占比较高、加速曲线趋于平缓（32 线程约 5.2 倍），而暴力基线这类秒级以上的重负载并行效率更高。

综上，BVH（求交算法层面）与多线程（并行层面）是两个正交的加速来源：前者把同场景的渲染时间从分钟级压到毫秒级、且几乎不随面数增长，后者在此之上再叠加数倍加速。两者共同使复杂网格的光线追踪渲染达到实用速度。

5 总结

本次作业完整实现了基于 BVH 加速的 Whitted 风格光线追踪器：从主光线生成、Möller-Trumbore 光线-三角形求交、AABB slab 包围盒测试，到 BVH 遍历，并额外补全了非 BVH 路径下的完整着色模型；在此之上做了系统优化——有序遍历 + 最近距离剪枝、SAH 构建、阴影 any-hit、OpenMP 多线程、抗锯齿与真实高光，并修正了阴影距离校验等正确性问题。优化后四千余面的 Bunny 场景 BVH 单线程约 0.24 s、32 线程约 40 ms。进一步的跨模型对照实验表明，在同一 32 线程下，相比逐三角形暴力求交，BVH 在 4968 至 34.6 万面的模型上取得 36 至 6588 倍加速，且渲染耗时几乎不随面数增长（单线程 0.24 $\rightarrow$ 0.32 s，面数 $\times 70$ 而时间仅 $\times 1.3$ ），凸显了层次包围盒在复杂场景下的必要性。通过本次实践，加深了对光线生成、解析法求交、空间数据结构加速、并行化以及局部光照（Phong + 阴影）等图形学核心概念的理解。